МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01. «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы
По дисциплине: «Архитектура суперкомпьютеров»
На тему: «Реализация алгоритма перевода чисел формата BF16 из двоичной системы счисления в десятичную и обратно»

Студент
группы 5130201/40101 _____ Храмов В.Д.

Преподаватель _____ Чуватов М.В.

«____» _____ 2026 г.

Санкт-Петербург
2026

1 Цель работы

Изучить формат представления чисел с плавающей точкой `bfloat16` (BF16), разработать и программно реализовать алгоритмы:

- перевода десятичного числа в его 16-битное представление BF16;
- обратного перевода 16-битного представления BF16 в десятичное число.

2 Теоретическая часть

2.1 Формат BF16

`bfloat16` — это 16-битный формат числа с плавающей точкой, использующий ту же ширину поля экспоненты, что и стандартный 32-битный формат `float` (IEEE 754 single precision), но с урезанной мантиссой. Формат состоит из трёх полей:

Знак (S)	Экспонента (E)	Мантисса (M)
1 бит	8 бит	7 бит

Смещение (bias) экспоненты равно 127. Значение нормализованного числа вычисляется по формуле:

$$value = (-1)^S \times 1.M \times 2^{(E-127)} \tag{1}$$

где $1.M$ — мантисса с неявной единицей впереди.

2.2 Специальные случаи

- $E = 0,\ M = 0$ — число равно ± 0;
- $E = 0,\ M \neq 0$ — денормализованное число: $value = (-1)^S \times 0.M \times 2^{-126}$;
- $E = 255,\ M = 0$ — $\pm\infty$;
- $E = 255,\ M \neq 0$ — NaN.

2.3 Диапазон и точность

Поскольку под экспоненту выделено столько же бит (8), сколько и в обычном `float32`, диапазон представимых значений BF16 совпадает с диапазоном `float32` и составляет примерно от 1.18×10^{-38} до 3.39×10^{38}. Плата за компактность формата — точность: из-за короткой (7-битной) мантиссы BF16 сохраняет лишь около 2–3 десятичных знаков точности, в то время как `float32` сохраняет около 7.

3 Алгоритм перевода

3.1 Из десятичной системы в BF16

1. Определить знак числа S и перейти к работе с его модулем.
2. Нормализовать число к виду $m \times 2^e$, где $1.0 \leq m < 2.0$, последовательным делением или умножением на 2.

3. Дробную часть нормализованной мантиссы $m - 1$ перевести в двоичный вид методом умножения на 2 с фиксированной точностью (*с запасом*, чтобы было по чему округлять).

4. Округлить полученную мантиссу до 7 бит по правилу *round-to-nearest-even* (округление к чётному в случае "ровно посередине"), учитывая перенос в экспоненту при переполнении мантиссы.

5. Вычислить смещённую экспоненту $E = e + 127$ и проверить границы диапазона (переполнение $\to \infty$, антипереполнение $\to 0$).

6. Собрать итоговые 16 бит: $S \,|\, E \,|\, M$.

3.2 Из BF16 в десятичную систему

1. Разбить 16 бит на поля S (1 бит), E (8 бит), M (7 бит).

2. Обработать специальные случаи (± 0, $\pm\infty$, NaN, денормализованные числа).

3. Перевести мантиссу M обратно в дробное десятичное число: $\sum_{i=1}^{7} bit_i \times 2^{-i}$.

4. Вычислить итоговое значение по формуле $(-1)^S \times (1 + M) \times 2^{(E-127)}$.

4 Реализация

4.1 Перевод целой и дробной части в двоичный вид

Перевод целой части выполняется классическим методом последовательного деления на 2 с накоплением остатков; перевод дробной части — методом последовательного умножения на 2:

```
private static String decimalToBinaryInteger(int num) {
    StringBuilder binary = new StringBuilder();

    if (num == 0) {
        return "0";
    }

    while (num > 0) {
        int remainder = num % 2;
        num /= 2;
        binary.append(remainder);
    }

    return binary.reverse().toString();
}
```

Листинг 1: Перевод целой части в двоичный вид

```
private static String decimalToBinaryFraction(double frac, int precision) {
    StringBuilder binary = new StringBuilder();
    int countPrecision = 0;

    while (frac != 0 && countPrecision != precision) {
        frac *= 2;
        if (frac >= 1) {
            binary.append(1);
```

```
            frac -= 1;
        } else {
            binary.append(0);
        }
        countPrecision++;
    }

    while (countPrecision != precision) {
        binary.append(0);
        countPrecision++;
    }
    return binary.toString();
}
```

Листинг 2: Перевод дробной части в двоичный вид

4.2 Нормализация

Нормализация объединяет целую и дробную части двоичного числа в одну строку `combined`. Позиция двоичной точки определяется по длине целой части и сохраняется в переменной `pointPosition`.

С помощью метода `indexOf('1')` находится позиция первой единицы в объединённой строке. Если единица отсутствует, число считается равным нулю, и выбрасывается исключение `IllegalArgumentException`.

Показатель степени вычисляется как разница между позицией двоичной точки и позицией первой единицы:

$$exponent = pointPosition - firstOne - 1.$$

Мантисса формируется из последовательности битов, расположенных после первой единицы, с помощью метода `substring(firstOne + 1)`. Таким образом, метод возвращает показатель степени и мантиссу нормализованного двоичного числа.

```
private static Normalized normalize(String binary, String binaryFraction) {
    String combined = binary + binaryFraction;
    int pointPosition = binary.length();

    int firstOne = combined.indexOf('1');
    if (firstOne == -1) {
        throw new IllegalArgumentException("");
    }

    int exponent = pointPosition - firstOne - 1;
    String mantissa = combined.substring(firstOne + 1);

    return new Normalized(exponent, mantissa);
}
```

Листинг 3: Нормализация через деление/умножение на 2

4.3 Округление мантиссы (round-to-nearest-even)

Так как исходная точность мантиссы (32 бита) значительно превышает целевые 7 бит BF16, отбрасываемые биты необходимо корректно округлять. Используется классическое правило: если бит-*guard* (8-й бит) равен 0 — округление не требуется; если он равен 1 и после него есть хотя бы одна единица — округление вверх; если это граничный случай

"ровно посередине"— округление производится к чётному значению последнего сохраняемого бита. При переполнении мантиссы (например, 1111111 округляется до 0000000) единица переноса добавляется к экспоненте.

```
private static Rounded roundMantissa(String mantissa, int exponent, int
    targetBits) {
    StringBuilder padded = new StringBuilder(mantissa);
    while (padded.length() < targetBits + 1) {
        padded.append('0');
    }

    String kept = padded.substring(0, targetBits);
    char guardBit = padded.charAt(targetBits);
    String remainder = padded.length() > targetBits + 1 ? padded.substring(
    targetBits + 1) : "";

    boolean roundUp;
    if (guardBit == '0') {
        roundUp = false;
    } else if (remainder.contains("1")) {
        roundUp = true;
    } else {
        char lastKeptBit = kept.charAt(kept.length() - 1);
        roundUp = (lastKeptBit == '1');
    }

    int keptValue = kept.isEmpty() ? 0 : Integer.parseInt(kept, 2);
    if (roundUp) {
        keptValue += 1;
    }

    int maxValue = (1 << targetBits) - 1;
    if (keptValue > maxValue) {
        keptValue = 0;
        exponent += 1;
    }

    String mantissaBits = toFixedLengthBinary(keptValue, targetBits);
    return new Rounded(exponent, mantissaBits);
}
```

Листинг 4: Округление мантиссы до 7 бит

4.4 Кодирование десятичного числа в BF16

Кодирование выполняется в несколько этапов. Сначала определяется знак числа и выделяется его абсолютное значение. Отдельно обрабатываются специальные значения: `NaN`, положительная и отрицательная бесконечность, а также ноль.

Для ненулевого конечного числа целая и дробная части преобразуются в двоичное представление. Для дробной части используется фиксированная точность в 32 двоичных разряда. Полученные части передаются в метод `normalize`, который определяет показатель степени и мантиссу нормализованного числа.

После нормализации мантисса округляется до 7 бит с помощью метода `roundMantissa`. К полученному показателю степени прибавляется смещение 127, используемое в формате BF16. Если после применения смещения показатель выходит за допустимый диапазон, значение обрабатывается как переполнение или антипереполнение.

Затем знак, показатель степени и мантисса объединяются в одно 16-битное значение с помощью побитовых операций. Знак занимает старший бит, показатель степени — следующие 8 бит, а мантисса — младшие 7 бит.

```
public static int encode(double value) {
    int sign = (value < 0) ? 1 : 0;
    double absValue = Math.abs(value);

    if (Double.isNaN(value)) {
        return (sign << 15) | (0xFF << 7) | 0x40;
    }
    if (Double.isInfinite(value)) {
        return (sign << 15) | (0xFF << 7);
    }
    if (absValue == 0.0) {
        return sign << 15;
    }

    int intPart = (int) absValue;
    double fracPart = absValue - intPart;

    String binaryInteger = decimalToBinaryInteger(intPart);
    String binaryFraction = decimalToBinaryFraction(fracPart, 32);

    Normalized norm = normalize(binaryInteger, binaryFraction);
    Rounded rounded =
        roundMantissa(norm.mantissa(), norm.exponent(), 7);

    int biasedExponent = rounded.exponent() + 127;

    if (biasedExponent >= 255) {
        return (sign << 15) | (0xFF << 7);
    }
    if (biasedExponent <= 0) {
        return sign << 15;
    }

    int mantissaBits = Integer.parseInt(rounded.mantissa(), 2);

    return (sign << 15) | (biasedExponent << 7) | mantissaBits;
}
```

Листинг 5: Кодирование double в BF16

4.5 Декодирование BF16 в десятичное число

Обратный перевод восстанавливает значение непосредственно по формуле стандарта, без использования встроенных средств Java для работы с битами `float`:

```
public static double decode(int bits) {
    bits = bits & 0xFFFF;

    int sign = (bits >> 15) & 0x1;
    int exponent = (bits >> 7) & 0xFF;
    int mantissa = bits & 0x7F;

    double signMultiplier = (sign == 0) ? 1.0 : -1.0;

    if (exponent == 0 && mantissa == 0) {
        return signMultiplier * 0.0;
```

```
    }
    if (exponent == 255) {
        return (mantissa == 0)
                ? signMultiplier * Double.POSITIVE_INFINITY
                : Double.NaN;
    }
    if (exponent == 0) {
        double mantissaValue = mantissaToDecimal(mantissa, 7);
        return signMultiplier * mantissaValue * Math.pow(2, -126);
    }

    double mantissaValue = mantissaToDecimal(mantissa, 7);
    int realExponent = exponent - 127;

    return signMultiplier * (1 + mantissaValue) * Math.pow(2, realExponent);
}
```

Листинг 6: Декодирование BF16 в double

5 Тестирование

Корректность реализации проверена набором модульных тестов, охватывающих: точно представимые значения, ноль, отрицательные числа, потерю точности, округление к чётному в граничных случаях, специальные значения (NaN, $\pm\infty$) и выход за границы диапазона (переполнение и антипереполнение). Результаты работы программы на контрольных значениях приведены в таблице 1 (формат поля битов: S EEEEEEEE MMMMMMM).

Таблица 1: Результаты кодирования и декодирования чисел в формате BF16

Исходное значение	Знак	Экспонента	Мантисса	Биты BF16	Декодированное значение
3.0	0	10000000	1000000	0 10000000 1000000	3.0
5.25	0	10000001	0101000	0 10000001 0101000	5.25
0.1	0	01111011	1001101	0 01111011 1001101	0.10009765625
-2.5	1	10000000	0100000	1 10000000 0100000	-2.5
100.0	0	10000101	1001000	0 10000101 1001000	100.0
0.0	0	00000000	0000000	0 00000000 0000000	0.0
65536.0	0	10001111	0000000	0 10001111 0000000	65536.0
10^{40}	0	11111111	0000000	0 11111111 0000000	$+\infty$
10^{-40}	0	00000000	0000000	0 00000000 0000000	0.0

Значение 0.1 демонстрирует ожидаемую потерю точности: из-за короткой 7-битной мантиссы BF16 не может точно представить это число, и после декодирования получается близкое, но не идентичное значение (0.10009765625). Значения 3.0, 5.25, 100.0 и 65536.0, напротив, представимы в BF16 точно, что подтверждает корректность работы алгоритма нормализации и округления.

6 Выводы

В ходе выполнения лабораторной работы был изучен формат чисел с плавающей точкой BF16 и самостоятельно реализован полный цикл перевода чисел между десятичной и двоичной системами счисления по этому формату — без использования готовых средств языка Java для работы с битовым представлением `float`.